

PRÁTICA — AUDITORIA E HARDENING DE UMA IMAGEM DOCKER

Sistema de Biblioteca — Auditoria, Correção e Validação de Vulnerabilidades

Objetivo

O processo será:

Construir → Escanear → Selecionar → Investigar → Corrigir → Reconstruir → Escanear novamente → Comparar

1. O que cada grupo deverá fazer

Cada grupo deverá:

1. Construir a imagem Docker do Sistema de Biblioteca.
2. Verificar se a aplicação está funcionando.
3. Executar um Security Scan.
4. Analisar as vulnerabilidades encontradas.
5. Selecionar:
 - **5 vulnerabilidades HIGH (abaixo disso, investigue todas)**
 - **5 vulnerabilidades MEDIUM (abaixo disso, investigue todas)**
 - **5 vulnerabilidades LOW (abaixo disso, investigue todas)**
6. Investigar a origem das 15 vulnerabilidades selecionadas.
7. Tentar corrigir as vulnerabilidades.
8. Construir uma nova versão da imagem.
9. Executar novamente o Security Scan.
10. Comparar os resultados da V1 com a V2.
11. Verificar se a aplicação continua funcionando.

O objetivo é demonstrar que vocês conseguem:

Identificar → Compreender → Corrigir → Verificar

2. Primeira etapa — Construção da imagem

Utilizem o repositório atual do Sistema de Biblioteca.

Exemplo:

```
docker build -t biblioteca-grupo1:v1 .
```

Substituem **grupo1** pelo número do grupo.

Verifiquem se a imagem foi criada:

```
docker images
```

Executem a aplicação:

```
docker run --rm biblioteca-grupo1:v1
```

Antes de continuar, confirmem:

- A imagem foi construída?
- O container iniciou?
- A aplicação funciona?
- O menu aparece corretamente?
- As principais funcionalidades estão funcionando?

Regra

Antes de começar o hardening, a aplicação precisa estar funcionando.

Isso será importante para a comparação entre V1 e V2.

3. Segunda etapa — Primeiro Security Scan

Utilizem o Docker Scout para analisar a imagem:

docker scout cves biblioteca-grupo1:v1

Analise o resultado completo.

Não comecem corrigindo imediatamente.

Primeiro, procurem compreender:

- Quantas vulnerabilidades foram encontradas?
 - Qual é a severidade?
 - Qual pacote está afetado?
 - Qual versão está instalada?
 - Qual é a versão corrigida?
 - De onde vem o pacote?
 - Ele pertence à aplicação?
 - Ele pertence à imagem base?
 - Existe uma atualização disponível?
-

4. Registro do primeiro Scan

Preencham:

Informação	Resultado
------------	-----------

Imagem

Critical

High

Medium

Low

Tamanho

Imagem
base

Atenção

As vulnerabilidades **CRITICAL** também deverão ser analisadas.

Entretanto, a seleção obrigatória para investigação será:

5 HIGH + 5 MEDIUM + 5 LOW

Se o scanner apresentar vulnerabilidades CRITICAL, registrem-nas e investiguem-nas também quando possível.

5. Terceira etapa — Seleção das vulnerabilidades

Cada grupo deverá escolher:

5 vulnerabilidades HIGH

Escolham cinco vulnerabilidades classificadas como:

HIGH

5 vulnerabilidades MEDIUM

Escolham cinco vulnerabilidades classificadas como:

MEDIUM

5 vulnerabilidades LOW

Escolham cinco vulnerabilidades classificadas como:

LOW

Total:

15 vulnerabilidades investigadas por grupo.

6. Como escolher as vulnerabilidades

Não escolham simplesmente as primeiras 15 vulnerabilidades da lista.

Procurem selecionar vulnerabilidades que permitam uma investigação.

Para cada vulnerabilidade, descubram:

1. Qual é o identificador da vulnerabilidade?

Exemplo:

CVE-2026-81642

2. Qual pacote está afetado?

Exemplo:

unbound

3. Qual versão está instalada?

Exemplo:

1.26.0-...

4. Qual é a versão corrigida?

Exemplo:

1.26.1-0+deb13u1

5. De onde o pacote está vindo?

Por exemplo:

Imagem base Debian

ou:

Dependência Python

6. Existe uma correção disponível?

7. É possível aplicar a correção sem quebrar a aplicação?

7. Investigação de uma vulnerabilidade

Para cada vulnerabilidade escolhida, preenchem:

Informação	Resposta
CVE	
Severidade	
Pacote afetado	
Versão instalada	
Versão corrigida	

Origem

Correção
disponível?

Correção aplicada?

Exemplo

Uma vulnerabilidade pode aparecer no scanner assim:

CRITICAL CVE-2026-81642

Affected range: <1.26.1-0+deb13u1

Fixed version: 1.26.1-0+deb13u1

A investigação deverá descobrir:

Qual pacote?



Qual versão?



Onde esse pacote está?



Por que ele está na imagem?



Existe atualização?



Como atualizar?



A atualização resolve?

8. Quarta etapa — Tentar corrigir

Depois de investigar as vulnerabilidades, tentem aplicar as correções.

As correções podem envolver, por exemplo:

Atualizar a imagem base

FROM python:3.13-slim

Atualizar pacotes do sistema

```
RUN apt-get update && \  
    apt-get upgrade -y && \  
    rm -rf /var/lib/apt/lists/*
```

Atualizar dependências Python

Alterando:

requirements.txt

e instalando novamente:

```
RUN pip install --no-cache-dir -r requirements.txt
```

Melhorar o Dockerfile

Vocês também podem encontrar problemas relacionados a:

- execução como root;

- pacotes desnecessários;
- arquivos desnecessários;
- dependências desatualizadas;
- imagem base;
- arquivos copiados para a imagem;
- configuração inadequada.

A correção deve ser baseada na investigação da vulnerabilidade.

Não alterem o Dockerfile aleatoriamente apenas para tentar diminuir o número mostrado pelo scanner.

9. Quinta etapa — Construir a V2

Depois das alterações, construam uma nova versão:

```
docker build --no-cache -t biblioteca-grupo1:v2 .
```

Verifiquem:

```
docker images
```

Agora testem a aplicação:

```
docker run --rm biblioteca-grupo1:v2
```

Confirmem que o sistema continua funcionando.

10. Sexta etapa — Novo Security Scan

Executem novamente:

```
docker scout cves biblioteca-grupo1:v2
```

Compare os resultados com a V1.

Perguntem:

- As vulnerabilidades escolhidas desapareceram?
 - Algumas permaneceram?
 - Alguma vulnerabilidade nova apareceu?
 - A quantidade total mudou?
 - O tamanho da imagem mudou?
-

11. Comparação V1 × V2

Preencham:

Indicador	V1 — Antes	V2 — Depois
Critical		
High		
Medium		
Low		
Tamanho		
Usuário		
Aplicação funcionando?		

12. Resultado das 15 vulnerabilidades

Agora façam uma análise específica das vulnerabilidades escolhidas.

#	CVE	Severidade	Corrigida?	Como foi corrigida?
1		HIGH		
2		HIGH		
3		HIGH		
4		HIGH		
5		HIGH		
6		MEDIUM		
7		MEDIUM		
8		MEDIUM		
9		MEDIUM		
10		MEDIUM		
11		LOW		
12		LOW		

13 LOW

14 LOW

15 LOW

13. Teste de funcionamento

Depois das correções, vocês deverão provar que a aplicação continua funcionando.

Testem as principais funcionalidades do Sistema de Biblioteca.

15. Ficha final de auditoria

Identificação

Integrantes:

Imagem V1

Nome:

Imagem base:

Tamanho:

Scan V1

CRITICAL: _____

HIGH: _____

MEDIUM: _____

LOW: _____

Vulnerabilidades investigadas

HIGH

1. _____
2. _____
3. _____
4. _____
5. _____

MEDIUM

6. _____
7. _____
8. _____
9. _____
10. _____

LOW

11. _____
12. _____
13. _____
14. _____
15. _____

Correções realizadas

Imagem V2

Nome:

Tamanho:

Scan V2

CRITICAL: _____

HIGH: _____

MEDIUM: _____

LOW: _____

16. Conclusão

Respondam:

1. Qual foi a principal origem das vulnerabilidades encontradas?

2. Quantas das 15 vulnerabilidades escolhidas foram corrigidas?

3. Quantas permaneceram?

4. Por que algumas não puderam ser corrigidas?

5. A aplicação continuou funcionando?

6. O que mudou entre V1 e V2?
